Project Development Phase
Code-Layout, Readability and Reusability

Date	10 July 2026
Team ID	SWTID-2026-6119
Project Name	Credit Card Approval Prediction System
Maximum Marks	4 Marks

Project Folder Structure:

The codebase is organized so that each concern - data, model training, and the web application - is kept in its own clearly named folder, matching standard ML project layout.

```
Project Development Phase/
  Dataset/
    credit_card_approval_dataset.csv
  Notebooks/
    EDA_and_Model_Training.ipynb
  Model/
    model.pkl
    encoders.pkl
    scaler.pkl
    feature_columns.pkl
  Flask/
    app.py
    templates/
      base.html
      index.html
      result.html
      404.html
    static/
      style.css
  Assets/
    (EDA and evaluation charts exported from the notebook)
```

Code Layout Principles Followed:

- **Separation of concerns:** data lives in Dataset/, exploratory analysis and model training logic lives in the notebook, trained artifacts live in Model/, and the web application lives in Flask/ - none of these mix responsibilities.
- **PEP8 style:** consistent 4-space indentation, snake_case function and variable names, and descriptive naming throughout app.py and the notebook (e.g. build_feature_vector, CATEGORY_OPTIONS).
- **Docstrings and comments:** app.py opens with a module-level docstring explaining its purpose, and every function (build_feature_vector, index, predict) has a docstring describing what it does.
- **Logging over print statements:** app.py uses Python's logging module to record model loading, successful predictions, and errors - suitable for real deployment, not just local debugging.
- **Exception handling:** the /predict route wraps model inference in try/except blocks, distinguishing between validation errors (bad input) and unexpected errors, so the app never crashes on bad input.

Reusability:

- **build_feature_vector() is a single reusable function** that any route (or future API endpoint) can call to turn raw form input into a model-ready feature vector - the encoding/scaling logic is not duplicated anywhere.

- **Model artifacts are decoupled from application code.** model.pkl, encoders.pkl, and scaler.pkl are loaded once at startup and can be replaced with a retrained version without touching app.py, as long as feature_columns.pkl stays consistent.
- **Templates use Jinja2 inheritance** (base.html) so the navigation bar and footer are written once and reused across index.html, result.html, and 404.html rather than being copy-pasted.
- **CATEGORY_OPTIONS and NUMERIC_FIELDS are defined once** as constants at the top of app.py and referenced both when rendering the form and when validating submitted data - avoiding hard-coded duplication.

Readability:

- Functions are short and single-purpose (index() only renders the form; predict() only handles submission and prediction).
- Variable names describe their content (form_data, feature_columns, X_scaled) rather than generic names like data1, x, temp.
- The notebook uses Markdown cells to explain the reasoning before each code cell (why mean imputation for numeric fields, why F1-Score was used to select the best model, etc.), so a reviewer can follow the thought process, not just the code.